

1 [30pt] System

Consider the unicycle 4-state model. The state $x = [p_x, p_y, v, \theta]^T \in \mathbb{R}^4$ contains the 2D position (p_x, p_y) , the speed v , and the heading θ of the vehicle. The control input $u = [\dot{v}, \dot{\theta}]^T \in \mathbb{R}^2$ contains the acceleration $\dot{v}$ and the turning rate $\dot{\theta}$. The dynamic equation is

$$\dot{x} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} u \quad (1)$$

1.1 [1pt] Is this system control-affine?

1.2 [5pt] Which description is most appropriate? Choose one from each row

Linear	Nonlinear
Continuous-Time	Discrete-Time
Time-invariant	Time-varying
Deterministic	Stochastic
Continuous-State	Discrete-State

1.3 [8pt] Linearize the system at $[p_x^r, p_y^r, v^r, \theta^r]^T$. Write down the resulting dynamics.

1.4 [8pt] Compute the linearized system dynamics at $[1, 1, 0, 0]^T$. Is the linearized system at this point controllable? Justify your answer.

1.5 [8pt] Under which condition on the parameters $p_x^r, p_y^r, v^r, \theta^r$ is the linearized system controllable? Find sufficient and necessary conditions.

2 [46pt] Model Mismatch

In this section, the controller imagines that the vehicle follows the unicycle 4-state model

$$x_u = \begin{bmatrix} p_x \\ p_y \\ v \\ \theta \end{bmatrix}, \quad u_u = \begin{bmatrix} \dot{v} \\ \dot{\theta} \end{bmatrix}, \quad (2)$$

where (p_x, p_y) is the position in the fixed world frame, v is the forward speed, θ is the heading, $\dot{v}$ is the longitudinal acceleration, and $\dot{\theta}$ is the turning rate. The actual rollout uses the dynamic bicycle model supplied in `hw1/vehicle_models.py`. Both transitions use forward Euler integration with time step $\Delta t = 0.01$ s. Unless otherwise stated, use

$$x_u(0) = \begin{bmatrix} 0 \\ 0 \\ 0.5 \\ 0 \end{bmatrix}, \quad p_g = \begin{bmatrix} 5 \\ 5 \end{bmatrix}, \quad T = 10 \text{ s.} \quad (3)$$

Here, $x_u(0)$ is the initial unicycle state, p_g is the goal position with components $p_{g,x}$ and $p_{g,y}$, and T is the rollout duration.

2.1 [15pt] Unicycle feedback controller. Define the goal-position error $e_p \in \mathbb{R}^2$, distance to the goal $\rho \in \mathbb{R}_{\geq 0}$, desired heading θ_d , and wrapped heading error e_θ by

$$e_p = \begin{bmatrix} p_{g,x} - p_x \\ p_{g,y} - p_y \end{bmatrix}, \quad \rho = \|e_p\|_2, \quad (4)$$

$$\theta_d = \tan^{-1} \left(\frac{e_{p,y}}{e_{p,x}} \right), \quad e_\theta = \text{wrap}(\theta_d - \theta) \in [-\pi, \pi). \quad (5)$$

Here, θ_d is the angle of the position-error vector measured from the positive x -axis. The operator $\text{wrap}(\phi)$ maps an angle ϕ to $[-\pi, \pi)$. In code, compute this operation with the provided `wrap_angle()` function; do not implement a separate angle-wrapping rule.

Let $k_v > 0$ be the velocity-feedback gain, $k_\theta > 0$ be the heading-feedback gain, $v_{\max} > 0$ be the maximum desired speed, and v_d be the desired speed defined by

$$v_d = \min(v_{\max}, \rho) \max(0, \cos e_\theta), \quad (6)$$

Implement the two controller calculations

$$u_u = \begin{bmatrix} \dot{v} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} k_v(v_d - v) \\ k_\theta e_\theta \end{bmatrix}. \quad (7)$$

Complete `unicycle_control` in `problem.py`. Use the default gains $(k_v, k_\theta) = (2.0, 2.5)$, run the unicycle model, and submit the XY trajectory and every state-versus-time plot. Explain the purpose of each gain and of the factor $\max(0, \cos e_\theta)$. The supplied return lines already clip the acceleration and turning rate. After computing the errors and desired speed, you only need to calculate `v_dot` and `theta_dot`; do not add another saturation operation.

2.2 [15pt] Pure-rolling bicycle command conversion. The dynamic bicycle state and control follow the Lecture 2 and SPARK order

$$x_b = \begin{bmatrix} X \\ Y \\ v_x \\ v_y \\ r \\ \psi \end{bmatrix}, \quad u_b = \begin{bmatrix} a_x \\ \delta \end{bmatrix}, \quad (8)$$

where (X, Y) is the position in the fixed world frame, v_x and v_y are the longitudinal and lateral body-frame velocities, $r = \dot{\psi}$ is the yaw rate, ψ is the heading, a_x is the longitudinal acceleration command, and δ is the front-wheel steering angle. Implement the initial-state conversion

$$x_b(0) = \begin{bmatrix} p_x(0) \\ p_y(0) \\ v(0) \\ 0 \\ 0 \\ \theta(0) \end{bmatrix}. \quad (9)$$

At each step, evaluate the Problem 2.1 controller using the virtual unicycle state $\tilde{x}_u$, defined by

$$\tilde{x}_u = \begin{bmatrix} X \\ Y \\ v_x \\ \psi \end{bmatrix}. \quad (10)$$

The actual rollout plant remains the six-state SPARK dynamic bicycle, whose lateral motion is generated by front and rear tire forces. For the purpose of converting the unicycle command only, replace those tire-force equations by the no-side-slip, pure-rolling approximation. Here, α_f and α_r denote the front and rear tire slip angles, and $l_f > 0$ and $l_r > 0$ denote the distances from the vehicle center of mass to the front and rear axles, respectively. The pure-rolling constraints are

$$\alpha_f = \alpha_r = 0 \implies v_y - l_r r = 0, \quad v_y + l_f r = v_x \tan \delta. \quad (11)$$

With wheelbase $L = l_f + l_r$, these constraints give

$$r = \dot{\psi} = \frac{v_x}{L} \tan \delta. \quad (12)$$

Therefore, if the shared controller returns $(\dot{v}, \dot{\theta})$, implement the provided command conversion

$$u_b = \begin{bmatrix} a_x \\ \delta \end{bmatrix} = \begin{bmatrix} \dot{v} \\ \text{sat}_{\delta_{\max}} \left(\tan^{-1} \left(\frac{L \dot{\theta}}{v_x} \right) \right) \end{bmatrix}. \quad (13)$$

Here, $\delta_{\max} > 0$ is the steering-angle limit and $\text{sat}_{\delta_{\max}}(z) = \min(\delta_{\max}, \max(-\delta_{\max}, z))$.

The inverse tangent is taken on its principal branch, and the conversion is used on the forward-motion domain $v_x > 0$. The acceleration command already satisfies its limit from Problem 2.1. Use the pure-rolling relation only for the command conversion: do not replace the supplied tire-force bicycle transition used for the actual rollout.

Complete `bicycle_initial_state` and `bicycle_control`, run the bicycle model, and submit the XY trajectory and every state-versus-time plot. Explain the tracking error caused by model mismatch. Hint: relate the error to the bicycle's lateral velocity v_y and yaw rate r .

2.3 [16pt] Two-parameter performance analysis. Use the shared controller for both models and evaluate all nine gain pairs in

$$k_v \in \{1.5, 2.0, 2.5\}, \quad k_\theta \in \{1.5, 2.5, 3.5\}. \quad (14)$$

At matching sample indices k , corresponding to time $t_k = k\Delta t$, define the unicycle–bicycle position mismatch $e_p^{\text{mm}}(k)$ and orientation mismatch $e_\theta^{\text{mm}}(k)$ by

$$e_p^{\text{mm}}(k) = \left\| \begin{bmatrix} p_{x,k} \\ p_{y,k} \end{bmatrix} - \begin{bmatrix} X_k \\ Y_k \end{bmatrix} \right\|_2, \quad e_\theta^{\text{mm}}(k) = |\text{wrap}(\theta_k - \psi_k)|. \quad (15)$$

Here, $(p_{x,k}, p_{y,k}, \theta_k)$ are components of the unicycle state at t_k , while (X_k, Y_k, ψ_k) are components of the bicycle state at the same time. Use the provided `wrap_angle()` function for the angular difference.

Run the provided gain study and submit exactly two figures: (i) position error versus time and (ii) orientation error versus time. Each figure should show all nine gain pairs with a readable legend. Analyze how k_v and k_θ affect the transient and final errors, and discuss whether one gain pair is uniformly best for both errors.

3 [24pt] True or False

- 3.1 [3pt] There is a unique equilibrium point for any system.
- 3.2 [3pt] For linear systems, local stability implies global stability.
- 3.3 [3pt] A stable system is bounded.
- 3.4 [3pt] A stable system converges to its equilibrium.
- 3.5 [3pt] Adding additional actuators to a system can always improve its controllability.
- 3.6 [3pt] Unobservability occurs due to system stochasticity.

3.7 [3pt] For a Markov process, the future state depends on the past state even when the current state and control are given.

3.8 [3pt] For any nonlinear system with known model and known initial state, the future can be fully predicted by simulation.